Gene Prioritization Tool Documentation

1. Project overview

This project is a phenotype-driven gene prioritization pipeline with a desktop GUI written in python language. The program takes a phenotype entered by the user, retrieves related PubMed articles, extracts MeSH disease/chemical terms, connects those terms to genes and Gene Ontology terms, scores the relationships, and ranks genes most strongly associated with the phenotype.

The project has four main Python files:

- pipeline.py -core data pipeline and network visualization
- guigene.py - Tkinter graphical interface
- gene_prioritization.py -gene scoring and ranking
- forDB.py- SQLite database creation, updates, and caching

The system also depends on several local data files stored in a data/ folder.

2. Main workflow

The full workflow is:

1. User enters a phenotype in the GUI.
2. The pipeline queries PubMed using the phenotype as a MeSH term.
3. PMIDs and associated MeSH terms are downloaded.
4. MeSH terms are filtered to D-terms using a local MeSH reference file.
5. D-term scores are calculated using publication overlap statistics.
6. PubMed IDs are linked to human genes through gene2pubmed.
7. Genes are linked to GO terms through gene2go.
8. GO-path scores are calculated.
9. Genes are ranked using phenotype, D-term, GO and gene paths.
10. Results are cached in SQLite and displayed in the GUI.
11. Optional network visualization can be generated for a selected gene.

3. Required libraries

3.1 External libraries to install

This command can be used to install required libraries:

pip install biopython pandas matplotlib networkx joblib pandarallel

3.2 Standard library modules used

These are built into Python and do not need separate installation: time, datetime, re, csv, os, math, sqlite3, textwrap, threading, webbrowser, io, collections, urllib.error, http.client, concurrent.futures, tkinter

3.3 Python version

Recommended: Python 3.10 or newer

Tkinter usually comes with standard Python installs, but on some Linux systems it may need to be installed separately.

4. Required project files

The code expects the following structure:

The project folder contains several Python scripts and a database file: pipeline.py, guigene.py, gene_prioritization.py, forDB.py, and mainDB.db. It also includes a subfolder named data, which holds the files mesh_d_terms_final.csv, mesh_d_terms_with_nd.csv, gene2pubmed_human.txt, gene2go_filteredGO_human.txt, and gene_symbols_links.txt.

Data files description

- mesh_d_terms_final.csv -Contains MeSH D-term names used to filter phenotype-associated MeSH terms.
- mesh_d_terms_with_nd.csv -Contains D-term names and their global document counts (nd) for scoring.
- gene2pubmed_human.txt- Human-only gene-to-PubMed mapping.
- gene2go_filteredGO_human.txt -Human gene-to-GO mapping.
- gene_symbols_links.txt - Maps GeneID to gene symbol and NCBI link.

5. File documentation

5.1 pipeline.py

This file contains the main phenotype-to-gene pipeline and network visualization logic.

Imported functionality

It uses: BioPython Entrez and Medline for PubMed queries , pandas for table processing, SQLite for storage, networkx and matplotlib for graph visualization, helper functions from forDB.py, ranking functions from gene_prioritization.py

fetch_pubmed_mesh(phenotypes, email, start_year=1940, step=2, retmax=9999)

Purpose

Queries PubMed for each phenotype and downloads PMID-MeSH term pairs.

Inputs

- phenotypes: list of phenotype strings
- email: email address required by NCBI Entrez
- start_year: first year for PubMed search
- step: year block size for querying
- retmax: max number of PubMed results per query

Process

- Iterates phenotype by phenotype
- Splits the PubMed search into year windows
- Searches PubMed using:
    - phenotype as a MeSH term
    - date publication range
- Fetches MEDLINE records in batches of 200
- Extracts:
    - PMID
    - MH MeSH terms
- Cleans MeSH strings

- Writes output CSV for each phenotype

Output

Returns a list of saved CSV file paths.

Notes

Includes retry logic for partial connection failures and rate-limit protection with sleep intervals.

map_phenotype_to_dterms(phenotype_csv, mesh_d_terms_file)

Purpose

Filters MeSH terms down to D-terms using a local MeSH dictionary.

Inputs

- phenotype_csv: CSV output from fetch_pubmed_mesh
- mesh_d_terms_file: file containing valid MeSH D-terms

Process

- Loads phenotype PMID-MeSH file
- Normalizes MeSH strings
- Extracts base MeSH term before /
- Keeps only terms present in D-term lookup
- Converts spaces to underscores
- Adds phenotype name
- Creates row IDs

Output

Creates and returns a CSV file:
<phenotype>_phenotype_dterms.csv

calculate_dterm_scores(input_csv, nd_file, max_mid, max_dtid)

Purpose

Builds the phenotype-to-D-term table and calculates D-term scores.

Inputs

- input_csv: phenotype D-term mapping file
- nd_file: D-term global count file
- max_mid: current max mid in database
- max_dtid: current max dtid in database

Process

For each phenotype–D-term pair it computes:

- inters: number of unique PMIDs shared by phenotype and D-term
- nm: number of unique PMIDs for the phenotype
- nd: total global count for the D-term
- unio: union count
  nm + nd - inters
- dscore:
  inters / nd

Then assigns new IDs:

- mid
- dtid

Output

Returns a DataFrame for insertion into the mterm table.

g2p_in_meshT(df_mterm)

Purpose

Links phenotype-related PubMed IDs to human genes using a gene2pubmed file.

Inputs

- df_mterm: phenotype/D-term table

Process

- Loads gene2pubmed_human.txt
- Keeps only TaxID 9606

- Explodes PMID lists into one PMID per row
- Merges phenotype PMIDs with gene2pubmed
- Collects unique gene IDs per phenotype/D-term record
- Counts number of genes

Output

Returns the same DataFrame with added columns:

- genes
- genenum

dtrem_g2go_partial2(phenotype_gene_df)

Purpose

Links genes from phenotype-associated D-terms to GO terms.

Inputs

- phenotype_gene_df: phenotype/D-term/gene DataFrame

Process

- Explodes gene lists
- Loads gene2go_filteredGO_human.txt
- Creates GO lookup table with:
    - gokey
    - goterm
    - description
    - category
- Merges genes with GO terms
- Builds D-term to GO mapping table

Output

Returns two DataFrames:

1. go_terms_df
2. dterm_go_df

evaluate_dterm_go(dterm_go_df)

Purpose

Calculates GO-level evaluation metrics and scores.

Inputs

- dterm_go_df: D-term to GO to gene mapping table

Process

For each phenotype D-term / GO pair it computes:

- gene: unique genes in the pair
- gogenes: number of genes linked to the GO term
- genenum: number of genes shared in this D-term/GO path
- genetot: total number of genes linked to the D-term
- gscore:

genenum / sqrt(gogenes * genetot)

Output

Returns the evaluated D-term/GO DataFrame.

visualize_gene_network(...)

Purpose

Creates a phenotype -chemical/D-term -GO term -selected gene network diagram.

Parameters

- phenotype
- mterm_df
- dterm_go_eval_df
- ranked_genes
- gene_name
- top_chemicals
- top_go_per_chemical

Process

- Selects one ranked gene
- Retrieves GO descriptions from database
- Filters to the chosen gene
- Computes path scores:

path_score = dscore * gscore

- Selects top chemicals and top GO terms
- Builds directed graph with NetworkX
- Draws graph using matplotlib

Node types

- phenotype node
- chemical/D-term nodes
- GO term nodes
- selected gene node

Edge labels

- phenotype -chemical uses dscore
- chemical - GO uses gscore
- GO - gene uses combined path strength

run_pipeline_for_phenotype(phenotype_input, top_n)

Purpose

This is the main end-to-end pipeline.

Steps performed

1. Normalize phenotype input
2. Query PubMed
3. Save PMID-MeSH CSV
4. Filter to D-terms

5. Compute D-term scores
6. Save mterm data to SQLite
7. Map PMIDs to genes
8. Map genes to GO terms
9. Evaluate GO paths
10. Save GO tables to SQLite
11. Load relevant phenotype records from DB
12. Rank genes
13. Cache ranked genes into phenotype_genes
14. Save ranked genes to a text file

Output

Intended to return ranked genes DataFrame.

Important issue

At the end of the function, the code says:

return ranked_genes,

Because of the trailing comma, this returns a tuple containing ranked_genes, not the DataFrame itself.

It should be:

return ranked_genes

This is a required fix.

5.2 guigene.py

This file provides the graphical user interface.

Main purpose

Allows a user to:

- input phenotype
- choose number of top genes
- run the pipeline

- display ranked genes in a table
- download results
- visualize a selected gene network

GUI components

Input frame

Contains:

- phenotype text entry
- top genes entry
- Run button
- status/error label
- animated spinner label

Result frame

Contains:

- title label
- result table
- download button
- rerun button
- network visualization settings

Key functions

normalize(text)

Replaces spaces with underscores and trims whitespace.

start_search()

Validates user inputs, starts spinner, and launches background threads:

- one for running the search
- one for spinner animation

run_search(phenotype_db, top_n)

- checks if phenotype results already exist in cache
- if cached, loads cached results
- otherwise runs the full pipeline
- displays results in GUI

show_results(df, phenotype)

Switches from input view to result view and fills the table.

go_back()

Returns user to the input screen and clears fields.

show_table(df)

Loads ranked genes into the Tkinter Treeview.

Displayed columns:

- Rank
- Gene
- Score
- Top Chemical
- Top GO
- NCBI link

on_click(event)

Double-clicking the NCBI link column opens the gene's NCBI page in a browser.

copy_cell(event)

Right-click copies the selected cell to clipboard.

run_visualization()

Loads ranked genes and relevant database rows, then calls visualize_gene_network().

download_file()

Saves currently displayed results as a tab-separated text file.

5.3 gene_prioritization.py

This file ranks genes based on contributions from phenotype-associated D-terms and GO terms.

Global setup

- Loads gene_symbols_links.txt
- Creates:
    - gene_name_lookup
    - gene_link_lookup

These convert numeric GeneIDs into:

- readable gene symbols
- NCBI links

prioritize_genes(mterm_df, dterm_go_eval_df, top_n=None)

Purpose

Computes final gene ranking.

Process

1. Merges D-term GO data with mterm scores using dtid
2. Computes per-path contribution:

contribution = dscore * gscore

3. Groups by gene
4. For each gene:
    - sums all path contributions into gene_score
    - finds strongest path
    - records dominant chemical and GO term
    - adds readable gene symbol and NCBI link
5. Sorts descending by total score
6. Normalizes score:

gene_score_norm = gene_score / max(gene_score)

Output columns

- gene_id

- gene
- gene_link
- gene_score
- dominant_dterm
- dominant_go
- gene_score_norm

print_ranked_genes(ranked_genes, top_n=20)

Prints ranked genes to console.

save_ranked_genes_csv(ranked_genes, output_file)

Writes ranked genes to CSV.

5.4 forDB.py

This file manages the SQLite database.

Main responsibilities

- connect to SQLite
- create required tables
- append or replace data
- update GO terms
- cache phenotype results

create_connection(db_file)

Creates SQLite connection.

create_mterm_table(db_file)

Creates mterm table:

mid INTEGER PRIMARY KEY
mterm TEXT
dterm TEXT
inters INTEGER
pmids TEXT
nm INTEGER
nd INTEGER

unio INTEGER
dscore REAL
dtid INTEGER

This table stores phenotype-to-D-term scoring information.

create_dterm_table(db_file)

Creates dterm table:

id INTEGER PRIMARY KEY
dtid INTEGER
dterm TEXT
gene TEXT
goterm TEXT
gokey INTEGER
genenum INTEGER
gogenes INTEGER
genetot INTEGER
gscore REAL

This stores evaluated D-term to GO relationships.

create_go_term_table(db_file)

Creates go_term table:

gokey INTEGER PRIMARY KEY
goterm TEXT UNIQUE
description TEXT
category TEXT

This stores GO term metadata.

create_all_tables(db_file)

Creates all main tables:

- mterm
- dterm
- go_term

get_max_mterm_ids(db_file)

Returns:

- maximum mid

- maximum dtid

Used for incremental ID assignment.

get_max_dterm_id(db_file)

Returns maximum id from dterm.

append_mterm_table(mterm_df, db_file)

Loads existing mterm, concatenates new rows, then replaces table.

append_dterm_table(dterm_df, db_file)

Loads existing dterm, concatenates new rows, then replaces table.

update_go_term_table(go_terms_df, db_file)

Only appends GO terms not already present in go_term.

insert_local_mterm_rows(mterm_df, db_file)

Alternative insert method that attempts to append only new local rows based on a composite key.

Gene cache functions

create_gene_cache_table(db_file)

Creates cache table:

id INTEGER PRIMARY KEY AUTOINCREMENT
mid_val INTEGER
phenotype TEXT UNIQUE
genes_txt TEXT

save_gene_cache(db_file, phenotype, mid_val, ranked_df)

Stores ranked genes as tab-separated text inside SQLite.

load_gene_cache(db_file, phenotype)

Loads cached ranked genes and reconstructs a DataFrame.

6. Database design summary

The SQLite database mainDB.db stores both intermediate and final results.

Tables

- mterm - phenotype/D-term associations and D-scores

- dterm - D-term/GO/gene associations and G-scores
- go_term - GO term descriptions
- phenotype_genes - cached ranked gene lists

This allows the tool to avoid reprocessing already analyzed phenotypes.

7. Scoring logic

7.1 D-term score

For each phenotype and D-term:

dscore = inters / nd

Where:

- inters = number of shared publications between phenotype and D-term
- nd = global number of publications for that D-term

This favors D-terms that strongly overlap the phenotype but are not too globally common.

7.2 GO score

For each D-term and GO term:

gscore = genenum / sqrt(gogenes * genetot)

Where:

- genenum = number of genes in the D-term/GO overlap
- gogenes = total genes associated with the GO term
- genetot = total genes associated with the D-term

This rewards GO terms with meaningful gene overlap while controlling for size.

7.3 Final gene score

Each phenotype - D-term -GO -gene path contributes:

contribution = dscore * gscore

The final gene score is:

gene_score = sum(all contributions for that gene)

Then scores are normalized:

gene_score_norm = gene_score / max(gene_score)

8. How to run the project

8.1 Install dependencies

pip install biopython pandas matplotlib networkx joblib pandarallel

8.2 Make sure required data files exist

Place all required files inside the data/ directory.

8.3 Run GUI

python guigene.py